

Spec-Driven Development with LLMs

A quick practical guide using a floor-plan-to-3D Python project

Goal: learn the workflow quickly, keep the LLM constrained, and make the repository - not the chat - the source of truth.

Example project

A Python tool reads a 2D apartment floor-plan image, detects structural geometry, generates a 3D model, and exports GLB/glTF for use in Godot.

Contents

1. SDD in one minute
2. Repository structure
3. The spec lifecycle
4. Writing a useful spec
5. Floor-plan-to-3D example spec
6. Prompting Claude: spec first, implementation second
7. Handling spec changes
8. Tests, failures, and infinite-loop protection
9. Git hooks and team sharing
10. Practical rules and anti-patterns
11. Fast checklist

1. SDD in one minute

Spec-driven development (SDD) means you define the desired behaviour and constraints before implementation. The LLM implements against that approved spec instead of inferring requirements from a long chat.

```
Requirement
|
v
Spec -> Plan -> Tests -> Implementation -> Verification -> Human review
```

Use this priority order:

```
Specification
v
Acceptance criteria
v
Tests
v
Implementation
```

The conversation is temporary. The repository is durable.

2. Repository structure

A simple structure is enough:

```
floor2gltf/  
|-- CLAUDE.md  
|-- README.md  
|-- pyproject.toml  
|-- .gitlab-ci.yml  
|-- .githubhooks/  
|   |-- pre-commit  
|   |-- pre-push  
|-- scripts/  
|   |-- setup-dev.sh  
|   |-- check.sh  
|   |-- test-full.sh  
|-- docs/  
|   |-- architecture/  
|   |-- decisions/  
|   |-- specs/  
|       |-- 001-basic-floorplan-to-glTF.md  
|-- src/floor2glTF/  
|-- tests/
```

CLAUDE.md: stable rules for how the agent works. **docs/specs/**: requirements for individual changes.
architecture/: how the system is structured. **decisions/**: ADRs for important architectural choices.

3. The spec lifecycle

Draft -> Approved -> Implementing -> Done -> Superseded

Recommended rules:

- **Draft:** freely edit the same spec.
- **Approved / Implementing:** update the same spec for refinements; record material changes.
- **Done:** avoid silently rewriting history. Create a new spec for a materially new capability.
- **Superseded:** keep the old spec for history and point to the replacement.

Git already versions files. Avoid filenames such as `final-v2-final.md`.

4. Writing a useful spec

A good spec is short enough to understand quickly and precise enough that the LLM does not invent behaviour.

Core sections

- **Summary:** what change is being made.
- **Problem:** why it is needed.
- **Goals:** what must work.
- **Non-goals:** what is explicitly out of scope.
- **Inputs / outputs:** API, CLI, files, formats.
- **Assumptions:** anything the implementation is allowed to rely on.
- **Processing / behaviour:** expected flow.
- **Acceptance criteria:** concrete Given / When / Then behaviours.
- **Testing:** what proves the feature works.
- **Open questions:** unresolved product decisions.

Do not over-specify classes or methods unless the architecture itself is a requirement. Describe intent and constraints first.

5. Floor-plan-to-3D example spec

Spec: 001 - Basic floor plan to GLB

Status: Draft **Output:** GLB **Language:** Python **Target:** Godot

Summary

Read a 2D apartment floor-plan image, detect basic structural walls, convert the layout to metric 3D geometry, and export a GLB file.

Goals

- Read PNG/JPEG.
- Detect straight structural walls.
- Use a user-supplied scale.
- Generate floor and wall meshes.
- Export a valid GLB that imports into Godot.

Non-goals for V1

- Furniture recognition.
- Photorealistic materials.
- Curved walls or stairs.
- Multiple floors.
- Automatic scale inference.

Assumptions

- Top-down image.
- One apartment.
- High-contrast wall lines.
- Walls are vertical; floor is horizontal.
- Default wall height: 2.7 m.
- Default wall thickness: 0.15 m.

CLI example

```
floor2gltf apartment.png --output apartment.glb --scale 0.01 --wall-height 2.7
```

Pipeline

```
Image -> preprocessing -> wall detection -> 2D model -> topology cleanup  
      -> scale conversion -> mesh generation -> GLB export
```

Intermediate model

```
Apartment  
  walls: list[Wall]  
  rooms: list[Room]  
  doors: list[Door]  
  windows: list[Window]
```

```
Wall  
  start: Point2D  
  end: Point2D  
  thickness: float  
  height: float
```

Coordinate mapping

image X -> world X
image Y -> world Z
height -> world Y
units -> metres

Acceptance criteria

- Given a rectangular room and valid scale, four walls are generated at the correct size and configured height.
- Connected walls meet within the defined geometry tolerance.
- Given 100 px = 1 m, a 500 px wall becomes 5 m.
- An image with no usable floor-plan structure fails clearly and produces no production GLB.
- Generated GLB imports into the supported Godot version with Y as up and metre-based dimensions.

6. Prompting Claude: spec first, implementation second

Use two phases. Do not ask the agent to specify, implement, test, and redesign everything in one prompt.

Phase A - create/review the spec

We are using spec-driven development. Do not implement yet.

1. Read CLAUDE.md and relevant architecture/docs.
2. Inspect existing code and similar tests.
3. Create/update docs/specs/001-basic-floorplan-to-gltf.md.
4. Include goals, non-goals, assumptions, pipeline, data model, acceptance criteria, tests, risks, and open questions.
5. Show conflicts or ambiguities. Do not invent requirements.
6. Stop before implementation.

Phase B - implement the approved spec

The spec is approved. Implement it exactly.

- Re-read the spec and CLAUDE.md.
- Make the smallest change that satisfies it.
- Avoid unrelated refactoring.
- Add/update tests.
- Run targeted tests, then ./scripts/check.sh.
- Compare the final diff against every acceptance criterion.
- Report files changed, tests, results, deviations, and remaining risks.

Review the spec, tests, and Git diff. Do not use the chat transcript as the authoritative record.

7. Handling spec changes

If you discover that Task 3 needs adjustment after Task 2 is done, usually update the **same active spec**.

Update the same spec when:

- The change refines how the same requirement is fulfilled.
- You clarify an assumption, threshold, error case, or acceptance criterion.
- The feature is still being implemented.

Create a new spec when:

- The change introduces a new capability or workflow.
- It creates a new subsystem or independently deliverable feature.
- The old feature is already Done and the new behaviour is materially different.

```
docs/specs/  
  001-basic-floorplan-to-gltf.md  
  002-layout-correction-workflow.md  
  003-automatic-scale-detection.md
```

For material in-place changes, add a short revision note:

```
## Revision history  
### 5 September 2026  
Changed: wall thickness is now configured instead of auto-detected.  
Reason: reliable inference is outside the MVP.  
Impact on completed work: none.
```

8. Tests, failures, and infinite-loop protection

A failing test is evidence, not automatically the truth. The approved spec remains the primary behavioural reference.

failure -> diagnosis -> hypothesis -> targeted change -> targeted test

Retry policy for the LLM

- Read the full failure.
- State a root-cause hypothesis before changing code.
- Make one targeted change.
- Run the smallest relevant test.
- **After 3 failed targeted attempts: stop and diagnose.**

The stop report should classify the problem as one of:

- implementation defect
- test defect
- specification defect
- environment/tooling issue
- dependency/version issue
- unclear requirement

Hard guardrails

- Do not delete, skip, disable, or weaken tests just to get green.
- Do not increase timeouts without a root-cause reason.
- Do not suppress exceptions merely to satisfy tests.
- Stop on oscillation: fixing A breaks B, fixing B breaks A repeatedly.

If the test contradicts the approved spec, investigate the mismatch. The correct fix may be the test or the spec, not production code.

9. Git hooks and team sharing

Store shared hooks in the repository, but keep them thin. Put real checks in normal scripts that humans, Claude, and GitLab CI can all run.

```
.githubhooks/  
  pre-commit  
  pre-push  
scripts/  
  setup-dev.sh  
  check.sh  
  test-full.sh
```

.githubhooks/pre-commit

```
#!/usr/bin/env bash  
set -euo pipefail  
exec ./scripts/check.sh
```

scripts/setup-dev.sh

```
#!/usr/bin/env bash  
set -euo pipefail  
git config core.hooksPath .githubhooks  
chmod +x .githubhooks/* scripts/*.sh  
echo "Git hooks configured"
```

scripts/check.sh

```
#!/usr/bin/env bash  
set -euo pipefail  
uv run ruff check .  
uv run ruff format --check .  
uv run mypy src  
uv run pytest tests/unit -q
```

Each clone runs once:

```
./scripts/setup-dev.sh
```

Use the same canonical check in GitLab CI:

```
checks:  
  stage: test  
  script:  
    - ./scripts/check.sh
```

Hooks provide fast feedback. CI and protected branches are the authoritative enforcement layer, because local hooks can be bypassed.

10. Practical rules and anti-patterns

Good SDD habits

- Keep specs feature-sized and independently testable.
- Define non-goals to prevent scope creep.
- Make assumptions and uncertainty explicit.
- Prefer existing repository patterns.
- Constrain the diff: smallest change that satisfies the spec.
- Map important acceptance criteria to tests.
- Require a completion report.
- Use ADRs for durable architectural decisions, not ordinary feature details.

Avoid

- One giant spec for the entire product forever.
- Long conversational prompts becoming the only source of requirements.
- The LLM silently resolving product ambiguities.
- Specifying every class and method before understanding the codebase.
- Creating v2/final/final-final copies instead of using Git history.
- Letting the agent retry failing tests indefinitely.
- Relying on local hooks as security or policy enforcement.

11. Fast checklist

Before coding	Write or update the feature spec.
Spec	Goals, non-goals, assumptions, behaviour, acceptance criteria, tests, open questions.
Agent	Read CLAUDE.md + relevant architecture + existing code first.
Approval	Stop after spec/plan for non-trivial changes.
Implementation	Smallest diff; no unrelated refactor.
Tests	Targeted tests first; broader checks after.
Failure loop	Max 3 targeted attempts, then stop and diagnose.
Spec changes	Refinement = same active spec; new capability = new spec.
Git	Shared .githooks + setup-dev.sh + canonical scripts/check.sh.
CI	GitLab CI is authoritative.
Review	Spec -> tests -> diff -> acceptance criteria.
Done	Record results, deviations, and remaining risks.

If you remember only one thing: the spec defines the intended behaviour; the LLM proposes and implements; tests verify; Git and CI preserve and enforce.